KIKIRPA-CPP

Étape LightGlue

Rapport de réalisation

Développement du step, intégration au pipeline et stratégie de validation.

État de livraison
Implémentation disponible. Validation complète OpenCV / ONNX restant à effectuer.

Branche : lightglue-step

Révision de base : 96fbf03

Date : 21 octobre 2026

Sommaire

1. Résultat 3

2. Périmètre fonctionnel 3

3. Fichiers du lot 4

4. Correctifs du matcher 4

5. Choix d'intégration 5

6. Vérifications réellement effectuées 6

7. Validation restant à effectuer 7

8. Commandes de construction et d'exécution 8

9. Commandes Git manuelles après revue et validation 9

Lecture du document

Les contrôles réalisés sont distingués des vérifications restant à effectuer. Les commandes Git de la dernière section sont fournies pour une exécution manuelle.

1. Résultat

L'étape `LightGlueStep` est implémentée et raccordée à un parcours C++ import -> correspondances LightGlue ainsi qu'à un lanceur CLI dédié. Le code et les tests sont prêts à être examinés. La validation complète du build et de l'inférence reste à effectuer dans un environnement disposant d'OpenCV et d'ONNX Runtime, avec un modèle compatible.

Aucun commit, push, fetch, changement de branche ni installation de dépendance n'a été réalisé. Les anciens rapports d'audit ne sont pas modifiés. La modification locale préexistante de `LightGlueMatcher.h` (absence de saut de ligne final) a été conservée et est exclue des commandes d'ajout proposées.

2. Périmètre fonctionnel

- Lecture de `tiles` depuis le DataStore, avec le modèle courant `vector<TileImage>` de main.

- Identification des voisines immédiates droite/bas, sans doublon, avec ordre déterministe ligne/colonne.

- Conservation des indices d'origine, chemins et coordonnées de grille des deux images.

- Une session matcher chargée à la demande, réutilisée pour les paires et les relances ; recréation si le chemin du modèle change.

- Appel du matcher sur les images complètes. Le contexte overlap est transmis mais reste ignoré par le matcher existant ; aucune promesse de découpage de bande.

- Vérification du nombre de correspondances (4 par défaut), des tailles de vecteurs, des coordonnées, des scores et de la translation.

- Publication du vecteur complet sous `matches`, de type `PairMatches`, seulement après succès de toutes les paires.

- Échec explicite et diagnostic avec coordonnées de la paire ; suppression des anciennes sorties avant une nouvelle exécution.

Les sorties sont des correspondances et une estimation de translation. Elles ne sont ni des transformations RANSAC ni des positions globales. Aucun assemblage ou export de mosaïque n'a été ajouté.

3. Fichiers du lot

Fichier	Changement
include/Backend/steps/LightGlueStep.h	Contrats LightGlueInput, MatchedTile, TilePairMatches/PairMatches et classe dérivée de Step.
src/Backend/steps/LightGlueStep.cpp	Validation, voisinage, cycle de vie du matcher, exécution et publication atomique au niveau fonctionnel.
include/Backend/core/DataStore.h	Ajout de erase pour retirer une sortie périmée et include utility explicite.
src/Backend/core/Step.cpp	Définition de Step::option nécessaire à l'import.
include/Backend/core/PipelineBuilder.h	Paramètres modelPath/minMatches et déclaration du builder LightGlue.
src/Backend/core/PipelineBuilder.cpp	Construction ImportStep -> LightGlueStep ; ancienne signature refuse explicitement les matchers non implémentés.
src/Backend/core/Pipeline.cpp	DataStore neuf par run ; conversion des exceptions d'étape en échec journalisé.
src/Backend/steps/ImportStep.cpp	Remise à zéro de sa sortie et de ses diagnostics à chaque exécution.
src/Backend/algorithms/stitching/lightglue/LightGlueMatcher.cpp	Correctifs limités nécessaires aux images 16 bits, coordonnées et erreurs.
src/main_lightglue.cpp	CLI dossier + modèle + regex facultative ; résumé des correspondances ; codes 0/1/2.
CMakeLists.txt	Includes OpenCV explicites et cible stitching_lightglue.
cmake/algorithms.cmake	Disponibilité LightGlue explicite, test du step et test ONNX conditionnel.
tests/steps/test_lightglue_step.cpp	Tests du contrat avec matcher injecté et test d'intégration import TIFF.
tests/steps/test_lightglue_model.cpp	Test d'inférence avec modèle réel et translation synthétique connue.
docs/LIGHTGLUE_STEP.md	Contrats, configuration, limites, construction et exécution.
reports/lightglue-step/IMPLEMENTATION_REPORT.md	Rapport de réalisation et commandes manuelles.

4. Correctifs du matcher

1. Vérification des profondeurs 8/16 bits non signées et des canaux 1/3/4.

2. Normalisation vers float : division par 255 pour 8 bits, par 65535 pour 16 bits.

3. Dimensions adaptées aux multiples de 32, avec plafond de 3200 pixels par côté ; petite image éventuellement agrandie à 32.

4. Restauration des coordonnées avec des facteurs horizontal et vertical distincts, sans modifier la signature de l'en-tête existant.

5. Refus de dimensions batch incompatibles avant de former un tenseur unique.

6. Vérification des trois sorties batch avant accès.

7. Retour false sur images vides ou aucune correspondance retenue.

Le seuil du matcher reste 0.9. Les contrats de noms/tenseurs et la stratégie de sélection CoreML existants sont conservés. Le modèle ONNX effectif n'a pas été fourni : ces correctifs nécessitent encore une validation réelle, pas seulement syntaxique.

5. Choix d'intégration

`buildLightGluePipeline(settings)` construit un parcours indépendant du modèle historique Tuile/Pyramide. Les anciennes branches parser, tuile et frontend ne sont pas fusionnées. Le lanceur existant src/main.cpp n'est pas remplacé : utiliser la nouvelle cible stitching_lightglue.

L'ancienne fonction buildPipeline(settings, a, b), jusque-là vide, délègue uniquement pour matcher="lightglue" et lève une erreur pour les autres choix. Aucun faux pipeline SIFT réussi n'est conservé. Ce changement de comportement est intentionnel et à prendre en compte chez les futurs appelants.

Le DataStore est réinitialisé avant un run ; il n'y a pas de reprise de checkpoint dans ce parcours. Les sorties sont publiées seulement après succès global du step. Si l'import échoue, le résultat du run précédent n'est plus accessible via le DataStore du pipeline.

6. Vérifications réellement effectuées

Contrôle	Résultat	Limite
État Git initial et final, branche/HEAD	lightglue-step ; HEAD inchangé 96fbf03	Pas de commit ni réseau.
git diff --check	Réussi	Contrôle d'espaces et de conflits seulement.
CMake réel, BUILD_TESTING=ON, FETCHCONTENT_FULLY_DISCONNECTED=ON	Échec à find_package(OpenCV) : OpenCVConfig.cmake introuvable	Aucun build OpenCV/ONNX réel possible dans l'environnement détecté.
Syntaxe C++17 -Wall -Wextra -Werror de huit fichiers	Réussi	Déclarations OpenCV minimales temporaires, header JSON réel local. Ce n'est pas une compilation avec OpenCV réel.
Quatre groupes de tests d'orchestration	Réussis	Matcher contrôlé injecté et Mat minimal temporaire, sans pixels réels, TIFF ni ONNX.
Mêmes quatre groupes avec AddressSanitizer/UndefinedBehaviorSanitizer	Réussis, aucun diagnostic	Porte uniquement sur le chemin d'orchestration testé avec doubles.
Intégration TIFF via CTest	Non exécutée	OpenCV de développement absent.
Test réel modèle 8/16 bits	Non exécuté	ONNX Runtime/modèle non disponibles dans les emplacements vérifiés.
Tests algorithmiques préexistants	Non exécutés	Build réel bloqué en configuration.

Les huit fichiers vérifiés syntaxiquement sont LightGlueStep.cpp, Step.cpp, PipelineBuilder.cpp, Pipeline.cpp, ImportStep.cpp, main_lightglue.cpp et les deux nouveaux fichiers de tests. Le véritable LightGlueMatcher.cpp ne fait pas partie de cette vérification : ses dépendances ONNX/OpenCV complètes ne sont pas disponibles.

Les quatre groupes exécutés reprennent les fonctions du fichier de tests du step :

- grille 2x2 désordonnée, indices conservés, directions et absence de doublons, réutilisation/changement de modèle ;

- faux succès vide, false, exception, tailles incohérentes, NaN, point hors image, score invalide, relance propre et absence de publication partielle ;

- DataStore absent ou mal typé, grille vide, doublons, absence de voisins, image vide, profondeur invalide, coordonnées négatives et limite INT_MAX ;

- minimum de correspondances non entier/hors bornes, overlap invalide, chemin vide, option dupliquée/type déclaré incorrect et factory nulle.

Les déclarations de dépendances et le pilote limités ont été créés uniquement sous /private/tmp/kikirpa-lightglue-syntax. Ils ne sont pas ajoutés au dépôt. Ils ne remplacent ni le build réel ni le test avec un modèle.

Commandes de vérification utilisées :

```
cmake -S . -B /private/tmp/kikirpa-lightglue-step-check \
  -DBUILD_TESTING=ON -DFETCHCONTENT_FULLY_DISCONNECTED=ON

git diff --check
```

La vérification syntaxique utilise clang++ -std=c++17 -Wall -Wextra -Werror -fsyntax-only avec les déclarations temporaires et le header nlohmann_json déjà présent dans un autre dossier local. Aucune copie de cette dépendance n'est ajoutée au projet. Les tests d'orchestration ont été compilés une première fois normalement, puis avec -fsanitize=address,undefined et exécutés : quatre groupes PASS dans chaque cas.

7. Validation restant à effectuer

1. Rendre OpenCV de développement et nlohmann_json disponibles à CMake sans téléchargement implicite.

2. Configurer ONNX Runtime avec le module OpenCV dnn.

3. Fournir un modèle qui respecte l'un des deux formats décodés par LightGlueMatcher.

4. Construire les cibles et exécuter CTest, dont lightglue_step (inclut l'import TIFF réel).

5. Activer LIGHTGLUE_TEST_MODEL pour exécuter lightglue_model : translation de 128 px, tolérance 3 px, au moins quatre correspondances et 80 % de cohérence géométrique, en 8 puis 16 bits et avec dimensions non multiples de 32.

6. Vérifier un petit jeu d'images représentatif avant tout usage sur un corpus volumineux.

La qualité d'alignement, la compatibilité du modèle, la consommation mémoire et les temps d'inférence ne sont pas certifiés par cette livraison.

8. Commandes de construction et d'exécution

Remplacer les chemins d'exemple par les emplacements réellement disponibles. Aucune installation n'est effectuée par ces commandes.

```
cmake -S . -B build/lightglue-step \
  -DBUILD_TESTING=ON \
  -DFETCHCONTENT_FULLY_DISCONNECTED=ON \
  -DOpenCV_DIR="/chemin/lib/cmake/opencv4" \
  -Dnlohmann_json_DIR="/chemin/share/cmake/nlohmann_json" \
  -DONNXRUNTIME_ROOT="/chemin/onnxruntime" \
  -DLIGHTGLUE_TEST_MODEL="/chemin/modele.onnx"
cmake --build build/lightglue-step --parallel
ctest --test-dir build/lightglue-step --output-on-failure
./build/lightglue-step/stitching_lightglue "/chemin/images" "/chemin/modele.onnx"
```

9. Commandes Git manuelles après revue et validation

Ces commandes ne sont pas exécutées. La liste d'ajout exclut l'ancien audit, les fichiers temporaires et la modification préexistante de LightGlueMatcher.h.

```
git branch --show-current
git diff --check
git diff --stat

git add -- \
  CMakeLists.txt \
  cmake/algorithms.cmake \
  include/Backend/core/DataStore.h \
  include/Backend/core/PipelineBuilder.h \
  include/Backend/steps/LightGlueStep.h \
  src/Backend/core/Pipeline.cpp \
  src/Backend/core/PipelineBuilder.cpp \
  src/Backend/core/Step.cpp \
  src/Backend/steps/ImportStep.cpp \
  src/Backend/steps/LightGlueStep.cpp \
  src/Backend/algorithms/stitching/lightglue/LightGlueMatcher.cpp \
  src/main_lightglue.cpp \
  tests/steps/test_lightglue_step.cpp \
  tests/steps/test_lightglue_model.cpp \
  docs/LIGHTGLUE_STEP.md \
  reports/lightglue-step/IMPLEMENTATION_REPORT.md

git diff --cached --check
git diff --cached --stat
git diff --cached

git commit -m "feat: add LightGlue pipeline step"
git push -u origin lightglue-step
```

La dernière commande publie les changements : elle reste à lancer manuellement après les vérifications. Aucun résultat de test non exécuté ne doit être présenté comme réussi dans le message de commit ou la revue.